

Reexamining CGRA Memory Sub-system for Higher Memory Utilization and Performance

Lei Dai^{1,2}, Ying Wang^{*1,3,4}, Cheng Liu^{1,2}, Fuping Li^{1,2}, Huawei Li^{1,2,5}, Xiaowei Li^{1,2}

¹ SKLP, Institute of Computing Technology, CAS, Beijing, China

² University of Chinese Academy of Sciences, Beijing, China

³ CICS, Institute of Computing Technology, CAS, Beijing, China

⁴ Zhejiang Laboratory, Zhejiang, China ⁵ Peng Cheng Laboratory, Shenzhen, China

Email: {dailei19z, wangying2009, liucheng, lifuping20s, lihuawei, lxw}@ict.ac.cn

Abstract—Memory sub-system including the data memory and instruction memory in a typical CGRA often takes up considerable chip area and even dominates the performance of CGRAs. Observing the common phenomenon that instruction memory in CGRAs is highly under-utilized while the data memory is over-committed or vice versa, we propose a CGRA with unified memory architecture, UM-CGRA, to enable flexible on-chip memory sharing between data and instructions. Furthermore, PEs are also augmented to share data between neighbors working in parallel. Also, an on-chip memory sharing-aware mapping algorithm is developed to unleash the potential of the proposed architecture. Our experimental results show that UM-CGRA achieves 77% performance improvement on average over the baseline CGRA given the same amount of total on-chip memory. When setting the same performance goal, UM-CGRA achieves 10.7% chip area-saving and 28.6% energy efficiency improvement on average.

I. INTRODUCTION

Compared with general-purpose processors, domain specific architectures (DSAs) that promise significant performance and energy efficiency improvement are gaining increasing attention. Fully customised DSAs achieve the highest performance and energy efficiency, but they are usually difficult to adapt to the evolving applications and can suffer expensive non-recurring engineering costs. In contrast, FPGAs with fine-grained reconfiguration granularity enable rather flexible design for various DSAs, but the generality takes considerable hardware overhead in terms of power, timing, and chip area. Coarse-Grained Reconfigurable Architectures (CGRAs) with both moderate hardware overhead and reconfiguration capability present an attractive design trade-off between ASICs and FPGAs, and fit well with the requirements of many domain-specific workloads that may change over time [1]–[3].

CGRAs typically consist of an array of homogeneous processing elements (PEs) connected with nearest neighbor links and memory sub-systems, including global register files, instruction memory, data memory and etc. Part of the memory sub-systems, especially the data memory and instruction memory, can also be closely incorporated with each PE independently and even considered as part of PE in many CGRA designs. CGRAs inherently support operation-level reconfiguration for domain-specific tasks by pre-compiling the tasks for the CGRA [4]–[7]. Computation-intensive loops of tasks are extracted and transformed into software pipeline to

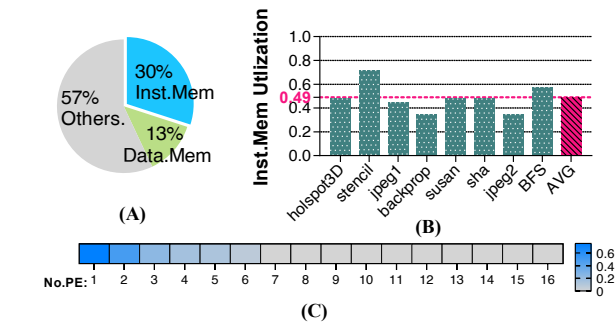


Fig. 1. Motivation experiments. (A) CGRA chip area breakup. (B) Low instruction memory utilization: 51% is idle on average. (C) Heatmap of imbalanced average data memory utilization: 1 or 2 PEs are very busy while 60% are almost empty.

be executed on CGRAs iteratively, typically using modular scheduling. By storing the scheduling results in the instruction memory and work-set data in data memory, CGRAs can invoke the iterative execution. With the computing dependency and hardware details fully exposed to the scheduler, the parallelism of the applications and hardware can be intensively explored, and significant performance can be achieved.

Despite the performance, CGRAs that have all the controlling details store and utilize data buffer for efficient data dependency resolving while ensuring high performance at the same time, which may require considerable on-chip buffer, i.e. the memory sub-system to fit diverse computing tasks. For instance, the memory sub-system in a typical 4*4 CGRA proposed in [8]–[10] takes up nearly half of the total area of CGRA, and 70% of the memory is consumed by CGRA instructions according to the statistics in Fig. I. However, we notice that both the instruction memory and data memory are underutilized at the same time. Particularly, half of the instruction memory is idle during the task execution, and the data memory utilisation is heavily imbalanced.

The instruction memory underutilisation can be mainly attributed to the following reason. Basically, the instruction memory needs to fully accommodate instructions generated for various target tasks to ensure iterative execution without a stall, so the instruction memory capacity is determined by the largest initiation interval (II) of the target tasks that may vary

dramatically. Like any other worst-case design, the instruction memory will suffer underutilisation for sure in most tasks. Unlike instruction memory, data memory capacity mainly depends on the average data reuse distance in a task, which varies not only across the different tasks but also different execution stages within a single task. Although less data memory does not necessarily fail the task mapping, it may cause dramatic performance degradation and larger II which will induce more instruction memory overhead. Basically, the requirements of instruction memory and data memory typically are not consistent, which can be a great opportunity for exploring the memory sharing between them.

To reduce the hardware overhead of the CGRA memory sub-system, prior works usually optimise the instruction memory and data memory independently. For the instruction memory underutilisation problem, different instruction or context file compression approaches are explored [11]–[17] to squeeze the redundancy within the instruction sequences and reduce the instruction memory requirements. However, the compression approaches cannot address the varied instruction requirements of different tasks. For the imbalanced data memory utilisation problem, existing works mainly seek to optimise the CGRA mapping algorithms [10], [18]–[21] and explore the data dependency more carefully with methods such as adopting a more complex memory hierarchy and optimising data layout without compromising the performance.

In contrast to the above independent optimisations, we propose to unify both the instruction memory and data memory for the sake of convenient runtime memory sharing and leverage an architecture-aware mapping algorithm to further improve the precious on-chip memory utilisation. Basically, with minor memory sharing implementation overhead, the proposed CGRA with unified memory architecture (UM-CGRA) can usually benefit when the data memory and instruction memory in a PE are not fully utilized at the same time. In addition, we also investigate the memory sharing between neighboring PEs and allow the PEs to share their in-PE memory as needed, which particularly alleviates the imbalance usage of data memory across the PEs as discussed in the previous paragraph. In summary, in order to address the severe underutilisation of the expensive on-chip memory of CGRAs, we revisit the memory sub-system of typical CGRAs by exploring different on-chip memory sharing architectures and corresponding mapping algorithms, and we mainly make the following contributions:

1) We observe that the memory sub-system of CGRAs takes up considerable chip area but suffers severe underutilisation. Particularly, the instruction memory and data memory in CGRA sub-memory system are not sufficiently utilized at the same time.

2) With the above observation, we develop a CGRA with Unified Memory Architecture (UM-CGRA) such that the instruction memory and data memory can be shared at runtime, which greatly improves the utilisation of the entire CGRA memory sub-system, especially the instruction memory. In addition, we also explore the memory sharing among neigh-

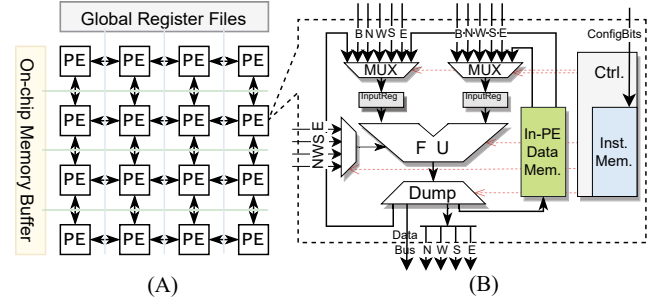


Fig. 2. Architecture of general CGRA with memory sub-system and a PE.

boring PEs to further address the imbalanced memory sub-system utilisation.

3) To unleash the potential of the proposed UM-CGRA, we develop a memory sharing aware mapping algorithm, which adopts a relaxation-and-tightening strategy to gradually fine tune the optimised memory partition between the data memory and instruction memory as well as the memory sharing among neighboring PEs. With a modular implementation, this strategy can be conveniently incorporated into existing CGRA mapping algorithms.

4) According to our experiments, the proposed UM-CGRA in combination with the shared memory aware mapping achieves 77% performance improvement on average under the same total amount of memory, 28.6% energy efficiency improvement and 10.7% area saving on average when achieving the same performance.

II. BACKGROUND AND RELATED WORK

A. Analysis of general CGRA architectures

As shown in Fig. 2, a typical CGRA contains a hierarchical memory subsystem. Except for the optional lower level memory outside PEs, a PE contains independent **instruction memory (IMEM)** and **work-set data memory (DMEM)** as higher-level memory for fast access. When it starts running, a loop kernel is extracted, scheduled and mapped to the CGRA. After the generated instructions are loaded to the IMEM of each PE over configuration buses, every PE executes the instructions in IMEM circularly until finished.

As analysed above, IMEM and DMEM occupy a considerable percentage of the total area but suffer from inconsistent low utilisation. This can fundamentally result from general CGRAs managing the IMEM and DMEM components independently as in the low-level cache of CPUs, which have very flexible instruction control flow and need specialised IMEM for efficiency. In contrast, a CGRA has only several static configurations executed circularly, and its IMEM and DMEM are both tightly coupled components. Thus, adopting independent memory management will cause memory space to be underutilized.

B. CGRA optimisation works

CGRA instruction optimisation. Existing instruction-reduction works for general CGRA mainly focus on reducing the instruction storage and fetching overhead or improving

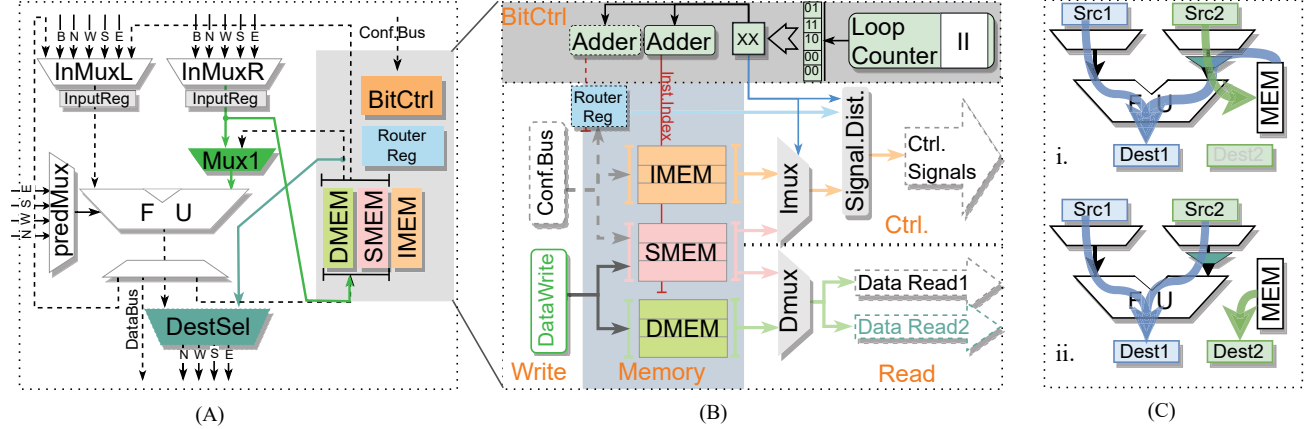


Fig. 3. UM-CGRA. (A) PE of UM-CGRA. (B) Detailed Unified Memory Architecture. (C) New non-conflict data paths of the PE.

both the area and energy efficiency. Methods proposed in [11], [12] reduce instructions by broadcasting or sharing among PEs. The works of [13], [14] compress and decompress the instructions with hardware. There are also works that compress the inactive instructions by dynamically fetching [15] or using a special encoding approach [16]. The latest process [17] partitions the context and only fetches the different parts of instructions to improve the efficiency with a tiny performance overhead. However, as they are all dedicated to reducing the overhead or improving efficiencies, they overlooked that instruction management can be highly correlated with task mapping.

CGRA Mapping. Many recent approaches have been proposed for mapping program loops onto CGRAs considering on-chip memory space. For simplicity, the 'mapping' stands for scheduling, placing, and routing an application onto a CGRA. Epimap [18] formulates mapping as the graph epimorphism problem eliminating dependencies using recomputing or routing with PEs. Regimap [19] further use in-PE DMEM to optimize and shrink schedule time. Memmap [20] is proposed to explicitly utilize the on-chip memory buffer aware of the varied access latency. RAMP [10] further introduces a powerful explicit rescheduling algorithm that comprehensively considers two levels of CGRA memory and neighbors' DMEM, but extra PEs are occupied. Crimson [9] and the work of [21] are the latest development. Crimson inherits the same memory management of RAMP. The later one extended all the aforementioned memory with global register files and developed the corresponding mapping algorithm. Other methods such as [22], [23] adopting classic optimisation heuristics and do not pay special attention to memory management.

Overall, one of their most significant common characteristics is to exploit more complex memory hierarchies to coordinate the conflicts between the memory space and the dependency solving of operations. Existing hardware of CGRA is almost drained for better mapping. Nevertheless, they seldom notice that the usage of DMEM is inconsistent with IMEM, and thus the potential of unified memory management is ignored.

III. UM-CGRA ARCHITECTURE

A. Overall Architecture

Key features of UM-CGRA. The proposed UM-CGRA possess two major features:

- 1) **IntraShare.** UM-CGRA shares parts of IMEM with DMEM, named with **Shared Memory (SMEM)**, to promote the utility of IMEM for computation.
- 2) **InterShare.** UM-CGRA shares SMEM among neighbor PEs utilizing idle links between PEs parallel to computation.

With IntraShare and InterShare, acquirable memory space for a PE in one cycle is not limited to DMEM but a unified set of memory, including the original DMEM, SMEM (part of the original IMEM), and its neighbors' SMEM. More importantly, all of them are modified from existing memory and link resources.

Overall Architecture of UM-CGRA. Fig. 3(A) depicts the overall architecture of a PE in UM-CGRA, including the two new added data paths and the overall Unified Memory Architecture (UMA), which is detailed in Fig. 3(B). Fig. 3(C) shows two situations of new non-conflict two data paths of PE in UM-CGRA.

In Fig. 3(A), the two newly added data paths required by InterShare are coloured green and blue. The green connections and Mux1 support reading data from local memory to FU concurrent with storing input data from the input 'InMUXR' to local memory, as in Fig. 3(C-i). These two paths are conflict in general PEs. Besides, the blue connections and DestSel are integrated to enhance PE with the ability to send two data to two different destinations, shown in Fig. 3(C-ii).

Other than the data paths, UMA's writing and reading ports are unaffected as originally seen from the outside of UMA. Detailed UMA in Fig. 3(B), the basis of IntraShare and InterShare, introduces the SMEM in addition to the original IMEM and DMEM and the redesigned controller logic. The SMEM, spilt from part of the original IMEM and DMEM, can store both instructions and work-set data. Additionally, two

TABLE I
MECHANISMS FOR BIT WIDTH MISMATCH

Situations	Measures
$DF\%(DW/2) < 10$	Ignoring exceeded bits.
$10 \leq DF\%(DW/2) < 16$	Exceeded bits are used as RouterReg.
$DF\%DW == 16$	Realigned to 32 bits.
$DF\%DW == 0$	An Inst. stores multiple work-set data.

multiplexers, Imux and Dmux, are added to select the sources for instructions and work-set data.

Controlling logic in UMA is also slightly modified to suit for IntraShare and InterShare. The BitCtrl indicates the source, including IMEM, SMEM or RouterReg, to get the current instruction. The RouterReg storing routing only instructions can be read and executed concurrently with computation instruction from IMEM or SMEM. Besides, idle instructions are also represented with bits in a register of BitCtrl to further release more free slots of IMEM for computation.

B. Mechanisms of UM-CGRA

For mechanisms of IntraShare, instructions indicated by BitCtrl will be read to the signal distributor (Signal.Dist.) to be analyzed and distributed in every cycle. Meanwhile, if local work-set data need to be accessed, one can be read from SMEM, and the other can be simultaneously read from DMEM indicated with a separate address signal. Control signals from BitCtrl are also sent to the Dmux to select the right sources.

To elaborate the mechanisms of InterShare, UM-CGRA can fetch, fuse and execute two non-conflict instructions with BitCtrl, RouterReg and signals distributor in the same cycle. The first one is a computation instruction from IMEM or SMEM. The second one is from RouterReg, performing the routing task along the coloured data paths in Fig. 3(A), which are depicted as green arrows in Fig. 3(C), without hindering the computation.

Bit width mismatch. In the case that the bit width of instruction is not equal to that of work-set data, two problems occur. First, work-set data can not fully utilize all the bits of an instruction slot. Second, the bit width of instruction in SMEM exceeds the width of the links between PEs when instruction needs to be shared with neighbors. For the first problem, we only consider the main mismatch cases that instruction width (IW) is not lower than that of work-set data (DW). According to the comparison between IW and DW, different measures, including ignoring, realigning, or being used as RouterReg are listed in Table. I. For the second problem, because mostly only part of the fields in instruction is useful, functions of instruction that can be transferred are restricted and only useful fields are transferred.

C. Micro-architecture Implementations

Shared memory(SMEM). SMEM can be considered as split from the original IMEM. Indices of both instructions and work-set data can cover the SMEM. Instructions or work-set data stored in SMEM will always be consecutive and start from one end, avoiding the intersection of indices. The part of SMEM serving as work-set data can also be linked with

TABLE II
SOURCE OF INSTRUCTION

BitCtrl	Idle	IMEM/SMEM	RouterReg
00	+		
01		+	
10			+
11		+	+

the original DMEM to be rotated. The bit width of SMEM is equal to IMEM because instructions are generally equal to or wider than work-set data. Two read ports are acquired since SMEM may be accessed by both reading instructions and reading work-set data in one cycle. Consequently, DMEM only needs one rather than two read ports to support the PE accessing two work-set data in one cycle. Besides, the write port of SMEM is still only one as it will not be simultaneously configured or written.

Bitmap controller (BitCtrl). BitCtrl is augmented from the original Loop-Counter with a register and two small Adders as depicted in the light grey area of Fig. 3(B). It indicates the sources of instructions and further collaborates with RouterReg to support loading two instructions in one cycle. Besides, memory slots originally occupied by idle instructions are also represented with two bits in the register. At every cycle, BitCtrl reads out 2 bits from the register indicated by the Loop-Counter and increases each adder to index instructions. These bits indicate four ways to fetch instructions as in Table. II. The '+' represents PE can fetch instruction from this source under this value of BitCtrl. For example, when 11 is read out, an instruction from IMEM/SMEM and the other one from RouterReg will be simultaneously loaded and executed.

Register for Routing only instructions (RouterReg). RouterReg is in charge of storing routing-only instructions and supporting InterShare. As shown in Table. III, RouterReg only stores four fields of the full instruction. And the four fields can be grouped as two source fields and two destination fields. The source fields are the InMuxR for input selection and the RightDataIndex for right source work-set data indexing. The destination fields are the outputDest for selecting one neighbor to be the output destination and the DestDataIndex for indexing work-set data to be stored. To save space, these fields can also be partially stored in the empty fields of full instruction.

Data Path Modifications Units. The MUX1 is a simple 2-to-1 multiplexer separated from the InMUXR, requiring an extra bit in instruction representation. The DestSel is a simple selector that selects one of the neighbors as the destination of the routing instruction from RouterReg. It is controlled by the OutputDest field in RouterReg.

D. Parameters

Size of each memory. The requirement of PE's local memory can be estimated in advance according to the target domains for a CGRA. In previous works, the sizes of IMEM are all assumed to meet the extreme demand, and the sizes of data memory are set in a heuristic way to balance the performance and overhead. In UM-CGRA, the sizes of IMEM,

TABLE III
INSTRUCTION FORMAT

Full_Inst:	InMuxR(3)	RightDataIndex (3)	OutputDest(2)	DestDataIndex (3)	Mux1 (1)	InMuxL(3)	Opcode(6)	Others(11)
Routing_Inst:	InMuxR	RightDataIndex	OutputDest	DestDataIndex	–	–	–	–

SMEM and DMEM must be balanced between benefits and costs. Separating more SMEM from IMEM will bring more flexibility and reduce the size of DMEM. Nevertheless, SMEM has the same bit number as IMEM, longer than DMEM, and has an extra reading port than IMEM. More SMEM will also bring about related control costs.

Therefore, the size of IMEM is set to be the average requirement of instruction to avoid the waste of ports of SMEM; that of DMEM can be unchanged or moderately reduced from the original; that of SMEM is the subtraction of the maximal requirement and the average requirement of instructions.

Instruction format and Overhead. The example common formats of a general CGRA full instruction with width and a RouterReg are shown in Table. III. RouterReg occupies less than half of the fields. Fields marked in bold are lengthen or newly added.

$$\text{overhead} = (\log_2(\text{SizeofDMEM} + \text{SizeofSMEM}) - \log_2(\text{SizeofDMEM})) * 2 + 3 \quad (1)$$

The instruction overhead of UM-CGRA contains three parts. First, indices of work-set data corresponding to the RightDataIndex and DestDataIndex need to be increased. The length is twice the subtract of the logarithm to the base 2 of the size of total work-set memory. Second, each instruction requires two bits in the BitCtl, equivalent to increasing the length of instruction by 2 bits. Third, the MUX1 requires an extra bit of instruction for control. In summary, the bits overhead for each instruction is as in (1).

IV. ALGORITHMS

To exploit the efficiency of UM-CGRA, a memory Sharing Aware Mapping (SAMap) algorithm is proposed. SAMap is compatible with existing CGRA mapping algorithms. Apart from memory management for IntraShare to avoid conflict, two key ideas are proposed to adapt to the features of UM-CGRA.

First, the main processes of SAMap adopt a two-stage relaxation-and-tightening strategy to be compatible with existing mapping algorithms without affecting their main bodies. The relaxation stage represents that the size of SMEM is firstly relaxed than the real for a feasible mapping tentatively, targeting performance tuning. The tightening stage represents that the mapping result is adjusted by utilizing the InterShare to satisfy the tightened real memory constraint.

Second, to exploit InterShare, a routing-only instruction can be overlapped with the other operation in one PE in the same cycle. The overlapping corresponds to the design of UM-CGRA that two new data paths can flow concurrently with computations.

Algorithm 1: Sharing_Aware_Mapping

Input: $DFG, CGRA, ID, SD, DD, MaxII, MaxRlx$
Output: $MappingGragh, Feasible_II$

```

1 Set_Mapping_Param( $DFG, CGRA, ID, SD, DD$ );
   $MII \leftarrow getMII(); Rst \leftarrow Null;$ 
2 for  $rlx$  from  $MaxRlx$  to 0 do
3    $Collect\_Mapping\_Info();$ 
4    $succ, ii, map \leftarrow Restrict(Mapping());$ 
5   if  $succ \ \& \ (rlx \neq 0)$  then
6      $succ, rst\_map \leftarrow Share\_Adjust(map, ii, rlx);$ 
7     if  $succ$  then
8        $Rst \leftarrow rst\_map, ii;$ 
9       if  $ii == MII$  then break;
10 return  $Rst$ 

11 def  $Share\_Adjust(map, curII, curRlx)$ :
12    $overPEs \leftarrow staticOverUsedPE();$ 
13   if  $isEmpty(overPEs)$  then return  $map$ ;
14   for  $pe$  in  $overPEs$  do
15     for  $npe$  in  $NeighborsOf(curPE)$  do
16       if  $canShareData(pe, npe)$  then
17          $map \leftarrow AdjustData(map, pe, npe);$ 
18       else if  $canShareInst(pe, npe)$  then
19          $map \leftarrow AdjustInst(map, pe, npe);$ 
20       else return  $Failed$ ;
21 return  $map$ 
```

A. Algorithm Framework Description

Algo. 1 depicts the SAMap, where the main body and the function ShareAdjust correspond to the relaxation stage and the tightening stage, respectively.

Relaxation Stage. Parameters are initialized in line 1. The MII stands for the minimal feasible II. In lines 2-10, the rlx , indicating how many virtual SMEM is relaxed, is iterated from the maximal($MaxRlx$) to 0 to find a temporary mapping result, which may be unacceptable under real constraints. In line 3, the mapping information is collected simultaneously to support deploying the restrictions in section. IV-B. After that, in line 4, those restrictions are applied to the imported mapping algorithm, $Mapping$, to find the temporary mapping result map while the II is ii and the relaxed SMEM is rlx . If it succeeds and the rlx is not 0, the tightening stage is activated by calling the $Share_Adjust$ (lines 5-6). Then, in lines 7-10, if the tighten stage succeeds and the ii achieves the optimal, the adjusted rst_map that satisfies the real constraints is returned. Otherwise, the program continues with the next loop until failed or finished.

Tightening Stage. The function $Share_Adjust$ is applied to adjust the temporary mapping result to satisfy the real memory

constraint. Firstly in lines 12-13, whether or where the memory requirement exceeds the real constrain is estimated. After that, if there are overused PEs(*overPEs*), every neighbor of each *overPEs* is checked to solve the exceeded memory requirement(lines 14-20). The checking includes the usability of ports, links, and SMEM slots. During this period, the InterShare of work-set data is firstly checked and conditionally adopted(lines 16-17), and then that of instructions is similarly done(lines 18-19). The adjusting functions will store the exceeded data in a neighbor and get it back when needed. If one of the exceeded PEs is not solved, the tightening stage fails. Otherwise, if all the *overPEs* are solved, the adjusted *map* is returned to the main processes.

B. Sub-functions

Mapping sub-function. To prove the efficiency, we realize SAMap based on and compared with RegiMap [19], which has been proven competitive in performance and compilation speed over other CGRA mapping algorithms, especially the simulated-annealing-based ones. It is selected because we only consider the existing basic memory level of IMEM and DMEM to avoid the influence of other factors such as lower-level additional work-set memory. REGIMap models the mapping process as a restricted maximum clique problem. Dependencies between operations can be solved via DMEM. In RegiMap, two operations with a dependency larger than 1 must be mapped to the same PE, which may impair the mapping results.

The *Mapping* need to be modified to suit for the overlapping of two operations in InterShare introduced above, which contains four conditions: 1) There is only one routing operation, and one non-routing instruction can be mapped on the same PE simultaneously. 2) There is no dependency between the two operations. 3) The destination of the two operations must be different. 4) The memory requirements are calculated as the sum of them.

Restrict sub-function. Some restrictions come from architecture of UM-CGRA, corresponding to the *Restrict* in SAMap, need to be realized. For IntraShare, conflict must be avoided when SMEM is allocated because it is shared by instructions and work-set data. Furthermore, InterShare is implemented utilizing originally under-utilized resources, including the original inputs and outputs, the invalid output links, the free slots of its SMEM, and the newly added parallel data path. Consequently, they need to be watched to decide whether they can be used. Besides, on the condition that not all types of instruction can be transferred between neighbors, types of instruction need to be identified and restricted. These restrictions can be directly implemented by guarding resources.

TABLE IV
EXPERIMENTS CONFIGURATIONS

Base					UMC-Intra/UMC-Intra+Inter				
No.Conf	DMEM	Req.Inst.(Avg)	Total		No.Conf	DMEM	SMEM	IMEM	Total
B1	2	12(8)	14		U1	2	2	6	10
B2	3	8(5)	11		U2	2	3	5	10
B3	4	6(4)	10		U3	2	4	4	10

V. EXPERIMENT RESULTS

A. Experiment Setup

Implementaion. A standard general CGRA from [8] is selected as the hardware baseline, referred to as **Base**. To evaluate the influence of each feature, UM-CGRA is separately implemented with only IntraShare, referred to as **UMC-Intra**, and with both IntraShare and InterShare, referred to as **UMC-Intra+Inter**. Corresponding mapping algorithms of Base, UMC-Intra and UMC-Intra+Inter are implied for brief. The designs are implemented in RTL and synthesized with SMIC 65nm technology, and run at 500 Mhz for energy estimation.

Benchmarks. We select 16 representative loops from cortex [24], machsuit [25], polybench [26], rodinia [27] and mibench [28] to test the performance, which are all computationally heavy and suitable for CGRA. The domains of these kernels range from ISP, security to generic computations and etc.

Compilation and Mapping. CCF [8] is adopted as our CGRA compilation framework, which contains a modified version of LLVM for front-end compilation, implements mapping algorithms as its passes, and simulates the performance using the cycle-accurate gem5 simulator. For the mapping process, the introduced competitive RegiMap is adopted to map kernels onto Base, referred to as **RIMap**. Meanwhile, SAMap based on RIMap will map kernels onto each implementation of UM-CGRA accordingly. All of them are based on CCF and share the same compilation front-end for a fair comparison.

Common CGRA Configurations. Most settings in Base, UMC-Intra and UMC-Intra+Inter are the same. PEs are connected in a 2D torus. Fixed-point operations are executed with 1-cycle latency. The memory bus is shared among PEs in a row. PEs can access on-chip memory such as on-chip scratchpad memory through load/store operations, requiring 2 instructions.

Experiment Configurations. Memory configuration for each experiment is as shown in Table. IV. The Base and UMC-Intra/UMC-Intra+Inter represent mapping kernels onto Base with RIMap and onto UMC-Intra and UMC-Intra+Inter with SAMap. For Base, 3 configurations, B1 to B3, increase the size of DMEM gradually. The *Req.Inst.* and the *Avg* in the quota is the max and average required IMEM size under each DMEM configuration, which will not continue decreasing clearly if DMEM increases larger than B3. For UMC-Intra and UMC-Intra+Inter, the size of DMEM and total memory of U1 to U3 are the same with the lowest DMEM and total memory requirements of Base. But different percentages of SMEM are set from U1 to U3. For hyper parameters of SAMap, *MaxRlx* is set to 3 and *MaxII* is set to 50.

B. Performance.

Fig. 4 shows the performance of UMC-Intra and UMC-Intra+Inter compared with Base related to the memory configurations, measured by *MII* normalized to *II*. In both Fig. 4(A) and Fig. 4(B), the rightmost group of columns are the average performance of all kernels.

First of all, although the performance of Base improves as DMEM grows from B1 to B3, the best performance is

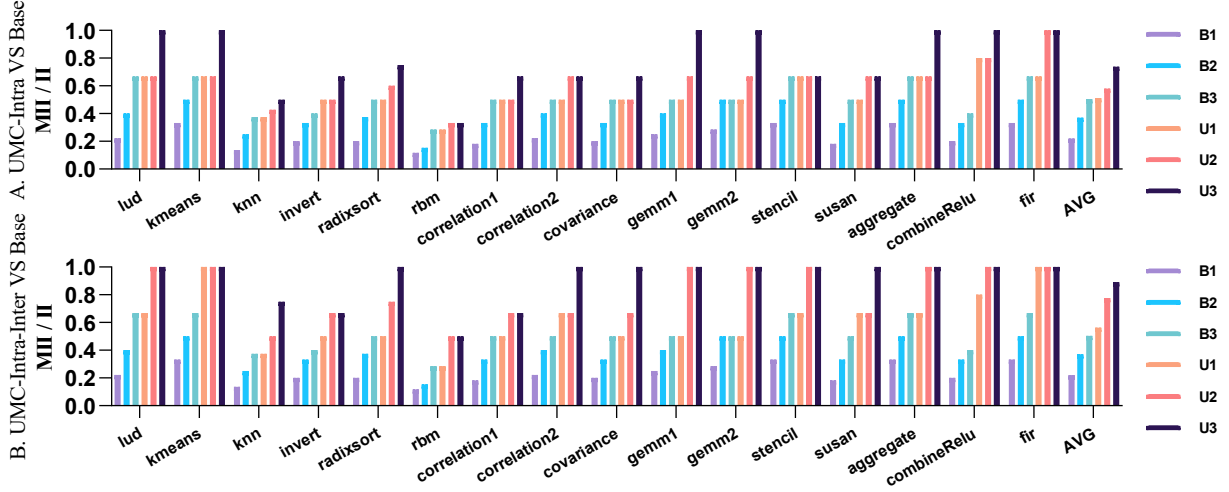


Fig. 4. Performance. (A) Performance of Base and UMC-Intra. (B) Performance of Base and UMC-Intra+Inter.

only 50% of the ideal. On the contrary, under the same total memory constraint, UMC-Intra+Inter can achieve an average performance improvement of 21% compared with the best of the UMC-Intra and 77% compared with the best of Base.

Influence of IntraShare. In Fig. 4(A), when comparing the average performance of UMC-Intra with Base, UMC-Intra under U1, U2, and U3 shows 2%, 16% and 28% improvement compared with the best performance of Base in B3 under the same total memory constraint. This comparison shows the effectiveness of IntraShare. Owing to the fact that original idle instruction memory is utilized to resolve computation dependencies, better performance is achieved.

Influence of InterShare. Next, when comparing the performance of the same kernels in Fig. 4(B) with that in Fig. 4(A) and Base, kernels under UMC-Intra+Inter achieve better performance. Under the same total memory constraint, the improvement is 21% compared with the best performance of the UMC-Intra in U3 and 77% compared with the best of Base in B3. Besides, the number of kernels that achieve optimal performance increases to 12 from 7 in UMC-Intra and 0 in Base. This demonstrates that InterShare is a more effective method to further resolve the performance by sharing neighbors' free SMEM adopting idle resources between PEs.

Influence of the percentage of SMEM. Furthermore, when comparing different configurations of UM-CGRA, performance improves from U1 to U3 in both Fig. 4(A) and Fig. 4(B). Although limited to the same total memory size, the increase in the percentage of SMEM from U1 to U3 will bring about more performance improvement because of the flexibility provided by SMEM.

Unsuitable Scenarios. There are also kernels such as knn, rbm and correlation1 that do not show significant improvement as other kernels. Because these kernels contain more load or store operations, they are more constrained by memory accessing than local memory assignment. Considering UM-CGRA is aimed at alleviating dependencies through more memory

resources, kernels constrained by other factors may not gain significant improvement. Nevertheless, more dependencies in kernels are between computational operations and can be more easily resolved with more local memory.

In conclusion, both InterShare and IntraShare of UM-CGRA are effective and will bring about corresponding performance improvement.

C. Energy Consumption and Area Overhead

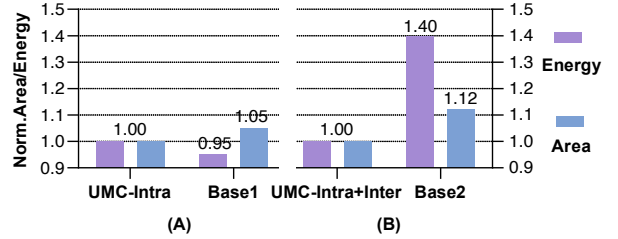


Fig. 5. Energy and Area targeting the same performance. (A) Situations under the same performance of UMC-Intra (B) Situations under the same performance of UMC-Intra+Inter.

To evaluate the area efficiency of UM-CGRA, Base is given enough resources to chase the same performance as UMC-Intra and UMC-Intra+Inter. UMC-Intra and UMC-Intra+Inter are configured under the memory configuration of U3. Resources of Base are gradually increased to chase the performance goals until just achieved. Then, the area and energy of Base are measured and compared with that of UM-CGRA.

Base1 in Fig. 5(A) and Base2 in Fig. 5(B) shows the area and energy when Base achieves the same performances with UMC-Intra and UMC-Intra+Inter, respectively. All the data are normalized to respective implementations of UM-CGRA. As shown in the results, Base1 needs 5% more area to achieve the same performance as UMC-Intra, and Base2 need 12% more area to be the same as UMC-Intra+Inter, demonstrating that UM-CGRA is area-efficient.

Besides, under the same performance goal, Base2 needs 40% more energy than UMC-Intra+Inter. Equivalently, the energy efficiency of UMC-Intra+Inter is 28.6% higher than Base. However, Base1 needs 5% less energy to achieve the same performance as UMC-Intra. This is because the complexity of controlling logic increases significantly more than the performance improvement with mere InterShare. Nevertheless, as InterShare additionally improves the performance but only brings about small hardware overhead, the energy efficiency of UMC-Intra+Inter is better than Base. In conclusion, although additional components introduced, UMC-CGRA is more efficient in both area and energy due to the significant performance improvement.

D. Compilation

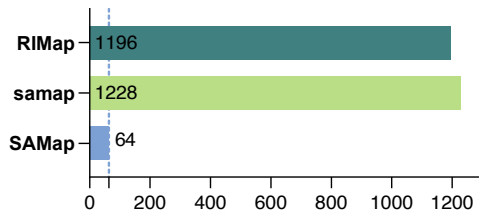


Fig. 6. Compilation Time(s)

Fig. 6 shows the average compilation time. The labels of RIMap, samap and SAMap, represent compilation conditions that raw RIMap under B3, SAMap with the same DMEM as B3 but given unlimited IMEM, zero SMEM and $MaxRlx = 0$, and SAMap under U3 with $MaxRlx = 3$.

The time of samap is 3% more than that of RIMap, which shows that the overhead of one-pass of SAMap compilation is small. Regarding the SAMap with fixed $MaxRlx$, the average compilation time decreases to 6% of the RIMap. This is resulted from that the based mapping algorithm can find a solution easier under a more relaxed restriction, and our architecture and strategies can provide more memory. As a result, although some overhead comes together, the total compilation time is decreased.

VI. CONCLUSION

We find that the highly under-utilized instruction or work-set data memory can be attributed to independent memory management, which limits the applicability of CGRAs. Based on the analysis, we propose the UM-CGRA to enable flexible on-chip memory sharing between data and instructions. Furthermore, PEs are also augmented to share data between neighbors working in parallel. Also, an on-chip memory sharing-aware mapping algorithm, SAMap, is developed to unleash the potential of the proposed architecture. On average, our experimental results show that UM-CGRA can achieve 77% performance improvements under the same total memory. What's more, given the same performance goal, UM-CGRA consumes 10.7% less chip area and improves energy efficiency by 28.6% on average.

VII. ACKNOWLEDGMENT

This paper is supported in part by the National Key Research and Development Program of China under grant 2018YFA0701502, and in part by the National Natural Science Foundation of China (NSFC) under grant No.(62090024, U20A20202, 61876173).

REFERENCES

- [1] Raghu Prabhakar et al. Plasticine: A reconfigurable architecture for parallel patterns. In *2017 ISCA*, pages 389–402. IEEE, 2017.
- [2] Yaqi Zhang et al. Sara: Scaling a reconfigurable dataflow accelerator. In *2021 ISCA*, pages 1041–1054. IEEE, 2021.
- [3] Zhaoying Li et al. Chordmap: Automated mapping of streaming applications onto cgra. *TCAD*, 2021.
- [4] Cheng Liu et al. Automatic nested loop acceleration on fpgas using soft cgra overlay. *arXiv preprint arXiv:1509.00042*, 2015.
- [5] Cheng Liu et al. Quickdough: A rapid fpga loop accelerator design framework using soft cgra overlay. In *FPT*, pages 56–63. IEEE, 2015.
- [6] Cheng Liu et al. A soft coarse-grained reconfigurable array based high-level synthesis methodology: Promoting design productivity and exploring extreme fpga frequency. In *FCCM*, pages 228–228. IEEE, 2013.
- [7] S Alexander Chin et al. Cgra-me: A unified framework for cgra modelling and exploration. In *ASAP*, pages 184–189. IEEE, 2017.
- [8] Shail Dave et al. Ccf: A cgra compilation framework. 2018.
- [9] Mahesh Balasubramanian et al. Crimson: compute-intensive loop acceleration by randomized iterative modulo scheduling and optimized mapping on cgras. *TCAD*, 39(11):3300–3310, 2020.
- [10] Shail Dave et al. Ramp: Resource-aware mapping for cgras. In *DAC*, pages 1–6. IEEE, 2018.
- [11] Yoonjin Kim et al. Resource sharing and pipelining in coarse-grained reconfigurable architecture for domain-specific optimization. In *DATE*, pages 12–17. IEEE, 2005.
- [12] Yoonjin Kim et al. Low power reconfiguration technique for coarse-grained reconfigurable architecture. *VLSI*, 17(5):593–603, 2009.
- [13] Nazish Aslam et al. Code compression and decompression for coarse-grained reconfigurable architectures. *VLSI*, 16(12):1596–1608, 2008.
- [14] Moo-Kyoung Chung et al. Efficient code compression for coarse grained reconfigurable architectures. In *ICCD*, pages 488–489. IEEE, 2012.
- [15] Yoonjin Kim et al. Dynamic context compression for low-power coarse-grained reconfigurable architecture. *VLSI*, 18(1):15–28, 2009.
- [16] Bernhard Egger et al. A space-and energy-efficient code compression/decompression technique for coarse-grained reconfigurable architectures. In *CGO*, pages 197–209. IEEE, 2017.
- [17] Zhongyuan Zhao et al. Similarity-aware architecture/compiler co-designed context-reduction framework for modulo-scheduled cgra. *Electronics*, 10(18):2210, 2021.
- [18] Mahdi Hamzeh et al. Epimap: Using epimorphism to map applications on cgras. In *DAC*, pages 1284–1291, 2012.
- [19] Mahdi Hamzeh et al. Regimap: Register-aware application mapping on coarse-grained reconfigurable architectures (cgras). In *DAC*, pages 1–10, 2013.
- [20] Shouyi Yin et al. Memory-aware loop mapping on coarse-grained reconfigurable architectures. *VLSI*, 24(5):1895–1908, 2015.
- [21] Zhongyuan Zhao et al. Towards higher performance and robust compilation for cgra modulo scheduling. *TPDS*, 31(9):2201–2219, 2020.
- [22] Bingfeng Mei et al. Dresc: A retargetable compiler for coarse-grained reconfigurable architectures. In *FPT*, pages 166–173. IEEE, 2002.
- [23] S Alexander Chin et al. An architecture-agnostic integer linear programming approach to cgra mapping. In *DAC*, pages 1–6, 2018.
- [24] Shelby Thomas et al. Cortesuite: A synthetic brain benchmark suite. In *IISWC*, pages 76–79. IEEE, 2014.
- [25] Brandon Reagen et al. Machsuite: Benchmarks for accelerator design and customized architectures. In *IISWC*, pages 110–119. IEEE, 2014.
- [26] Jeyhun Karimov et al. Polybench: The first benchmark for polystores. In *Technology Conference on Performance Evaluation and Benchmarking*, pages 24–41. Springer, 2018.
- [27] Shuai Che et al. Rodinia: A benchmark suite for heterogeneous computing. In *IISWC*, pages 44–54. Ieee, 2009.
- [28] Matthew R Guthaus et al. Mibench: A free, commercially representative embedded benchmark suite. In *WVC-4*, pages 3–14. IEEE, 2001.